Proyecto de Fundamentos de Ingeniería de Software

María Juliana Alzate Saavedra

Juan Esteban Cañon Solorza

Gerson David Cruz Rodriguez

Universidad Distrital Francisco José de Caldas

Facultad de Ingeniería

Fundamentos de Ingeniería de Software

Docente: Martín Delgado

Bogotá, Colombia

2026

Índice

Resumen **4**

1. Preguntas de Entrevista **5**

1.1. Funcionamiento General 5

1.2. Sistema de Inicio de Sesión y Roles 5

1.3. Inventario de Productos 5

1.4. Gestión Financiera 6

2. Requerimientos **7**

2.1. Requerimientos Funcionales 7

2.1.1. RF-01: Gestión de usuarios 7

2.1.2. RF-02: Gestión de productos e inventario 7

2.1.3. RF-03: Gestión de ventas 7

2.1.4. RF-04: Facturación 7

2.1.5. RF-05: Gestión financiera 8

2.1.6. RF-06: Reportes y resumen de ventas 8

2.2. Requerimientos No Funcionales 8

2.2.1. RNF-01: Servicio en la nube 8

2.2.2. RNF-02: Tiempos de carga 8

2.2.3. RNF-03: Seguridad 8

2.2.4. RNF-04: Portabilidad 8

2.2.5. RNF-05: Mantenibilidad 8

3. Historias de Usuario **9**

3.1. Autenticación 9

3.2. Gestión de Productos 9

3.3. Inventario 9

3.4. Ventas 9
3.5. Pago y Facturación 10
3.6. Reportes Diarios 10
3.7. Reportes Mensuales 10

4. Diagramas del Sistema 11
4.1. Diagrama de Casos de Uso 11
4.2. Profundización del Diagrama de Casos de Uso: Administrador 11
4.2.1. Consulta de Resumen de Ventas 11
4.2.2. Gestión de Productos e Inventario 12
4.3. Profundización del Diagrama de Casos de Uso: Cajero 13
4.3.1. Gestión de Cuenta de Venta 13
4.3.2. Emisión de Factura 13
4.3.3. Procesamiento de la Venta 14
4.4. Diagrama de Contexto 14
4.5. Diagrama de Secuencias 15
4.6. Diagrama de Entidad-Relación 17
4.7. Diagrama de Clases UML 20

5. Metodología de Desarrollo 22
5.1. Resumen de Metodologías Evaluadas 22
5.1.1. Scrum 22
5.1.2. Kanban 22
5.1.3. Extreme Programming (XP) 22
5.2. Metodología Implementada 23
5.2.1. Selección de Kanban 23
5.2.2. Roles del Equipo de Desarrollo 23

6. Arquitectura del Sistema 25

6.1. Decisión Arquitectónica 25
6.2. Organización Modular 25

7. Plan de Pruebas **26**
7.1. Pruebas Unitarias 26
7.2. Pruebas de Integración 27
7.3. Pruebas Funcionales 27
7.4. Pruebas de Usabilidad 28
7.5. Pruebas de Seguridad 29
7.6. Resumen del Plan de Pruebas 30

8. Mockup **31**

Referencias **33**

Resumen

El presente documento describe el proceso de análisis, diseño y planificación del desarrollo de un sistema de punto de venta (POS) orientado a pequeñas tiendas y minimarkets. El proyecto fue desarrollado en el marco del curso Fundamentos de Ingeniería de Software de la Universidad Distrital Francisco José de Caldas. Se documenta el levantamiento de requerimientos funcionales y no funcionales a través de una entrevista estructurada, la definición de historias de usuario, los diagramas de casos de uso, la arquitectura monolítica modular seleccionada, la metodología Kanban adoptada para la gestión del trabajo en equipo, el plan de pruebas con sus cinco tipos definidos, y el diagrama entidad-relación del modelo de datos. El sistema contempla dos roles principales: administrador y cajero, con módulos de gestión de productos, inventario, ventas, facturación y reportes financieros.

Palabras clave: sistema POS, ingeniería de software, Kanban, requerimientos, pruebas de software, arquitectura monolítica.

Preguntas de Entrevista

Como primer paso del levantamiento de requerimientos, el equipo elaboró un cuestionario estructurado para ser aplicado al cliente o usuario final del sistema. Las preguntas se organizaron en cuatro categorías: funcionamiento general del negocio, sistema de inicio de sesión y roles, inventario de productos, y gestión financiera.

Funcionamiento General

- ¿Qué tipo de negocio es? (tienda, minimarket, ferretería, restaurante, etc.)
- ¿Cuántos productos maneja aproximadamente?
- ¿Cuántos cajeros usarán el sistema al mismo tiempo?
- ¿La página será solo para uso interno o también para clientes?
- ¿Necesita que funcione solo en computadoras o también en celular/tablet?

Sistema de Inicio de Sesión y Roles

- ¿Cuántos roles existirán exactamente? (Administrador, Cajero, ¿otro más?)
- ¿El administrador puede ver y modificar todo?
- ¿El cajero puede editar productos?
- ¿El cajero puede eliminar productos?
- ¿El cajero tiene acceso a los resúmenes de ventas mensuales?
- ¿Necesita recuperación de contraseña?
- ¿De los dos roles del programa, quién es el encargado de ingresar la cantidad disponible del mismo producto?

Inventario de Productos

- ¿Qué datos se deben registrar de cada producto?
- ¿El ingreso de productos requiere el registro de un código de barras?
- ¿Los productos requieren de un número para poder identificarlos?

- ¿Se pueden editar productos después de creados?
- ¿Se pueden eliminar productos del inventario?
- ¿Se requiere conocer la cantidad en tienda física del mismo producto como dato del inventario?
- ¿Se debe generar una factura de la cuenta de venta? De ser así, ¿qué datos debe mostrar la factura?

Gestión Financiera

- ¿Qué información debe mostrar el reporte diario? (total vendido, número de ventas, productos más vendidos, ventas por cajero)
- ¿Se deben separar las ventas por método de pago (efectivo, transferencia, tarjeta)?
- ¿Cómo se registrarán los gastos mensuales y qué categorías incluirán?
- ¿El sistema debe calcular automáticamente ingresos, gastos y ganancia o pérdida mensual?
- ¿Se necesita comparación entre meses para analizar el rendimiento?
- ¿Se debe llevar control de caja diario? (monto inicial, arqueo, reposición, registro de diferencias)

Requerimientos

Los requerimientos del sistema se clasificaron en funcionales y no funcionales, siguiendo las convenciones estándar de la ingeniería de software (Sommerville, 2016). Los requerimientos funcionales describen los servicios que el sistema debe proveer, mientras que los no funcionales establecen las restricciones de calidad bajo las cuales debe operar.

Requerimientos Funcionales

Los requerimientos funcionales identificados a partir de la entrevista con el cliente y el análisis de las necesidades del negocio son los siguientes:

RF-01: Gestión de usuarios. La plataforma debe permitir el ingreso diferenciado a los cajeros y al administrador del supermercado mediante un sistema de autenticación seguro. Cada rol tendrá acceso únicamente a las funcionalidades asignadas según sus responsabilidades dentro del sistema.

RF-02: Gestión de productos e inventario. El sistema debe permitir al administrador crear, editar y eliminar productos. Cada producto contará con nombre, precio de venta, costo, imagen y categoría. El inventario deberá actualizarse automáticamente al realizarse una venta, y el administrador podrá realizar ajustes manuales con motivo registrado (merma, pérdida, conteo físico). El sistema emitirá alertas cuando el stock alcance el nivel mínimo configurado.

RF-03: Gestión de ventas. El cajero podrá crear cuentas de venta, agregar productos mediante búsqueda por código de barras, nombre o categoría, aplicar descuentos autorizados, seleccionar el método de pago (efectivo, tarjeta, transferencia) e ingresar el monto recibido para el cálculo automático del cambio.

RF-04: Facturación. El sistema debe generar automáticamente una factura con los datos de la compra al confirmar una venta. La factura incluirá listado de productos, cantidades, precios unitarios, subtotal, impuestos y total.

RF-05: Gestión financiera. El administrador podrá registrar los gastos del negocio por categorías y consultar reportes de ingresos y egresos mensuales y anuales. El sistema calculará automáticamente la ganancia o pérdida del período.

RF-06: Reportes y resumen de ventas. El sistema generará reportes diarios y mensuales con información sobre ventas brutas, descuentos aplicados, impuestos cobrados, ventas netas, número de transacciones, ticket promedio y comparativos entre períodos. Los reportes podrán filtrarse por cajero, método de pago y rango de fechas.

Requerimientos No Funcionales

Los requerimientos no funcionales establecen las condiciones de calidad del sistema:

RNF-01: Servicio en la nube. El sistema deberá ser desplegado en infraestructura de nube para garantizar disponibilidad y acceso remoto por parte del administrador.

RNF-02: Tiempos de carga. Las operaciones críticas del cajero (búsqueda de producto, registro de venta) deberán completarse en menos de dos segundos bajo condiciones normales de operación.

RNF-03: Seguridad. El sistema implementará autenticación por credenciales, cifrado de contraseñas, control de acceso por rol y protección ante accesos no autorizados a rutas restringidas.

RNF-04: Portabilidad. La interfaz deberá ser responsiva y funcional en computadoras de escritorio, tabletas y teléfonos móviles, adaptándose a diferentes tamaños de pantalla sin pérdida de funcionalidad crítica.

RNF-05: Mantenibilidad. La arquitectura modular del sistema deberá facilitar la incorporación de nuevas funcionalidades y la corrección de errores con un impacto mínimo sobre los módulos existentes.

Historias de Usuario

Las historias de usuario constituyen la expresión de los requerimientos desde la perspectiva del usuario final, siguiendo el formato estándar ágil: *"Como [rol], quiero [funcionalidad] para [objetivo]"* (Cohn, 2004).

Autenticación

- Como administrador, quiero iniciar sesión para gestionar el sistema.
- Como cajero, quiero iniciar sesión para registrar ventas.
- Como sistema, quiero validar credenciales para permitir acceso seguro.

Gestión de Productos

- Como administrador, quiero crear productos para agregarlos al sistema.
- Como administrador, quiero editar productos para actualizar su información.
- Como administrador, quiero eliminar productos para mantener el sistema organizado.
- Como administrador, quiero agregar nombre, precio, costo e imagen para identificar los productos.

Inventario

- Como administrador, quiero visualizar el inventario para controlar existencias.
- Como sistema, quiero actualizar el inventario automáticamente al realizar ventas.

Ventas

- Como cajero, quiero crear una nueva cuenta para iniciar una venta.
- Como cajero, quiero agregar productos a la cuenta para registrar la compra.
- Como cajero, quiero ver el total de la compra antes de cobrar.
- Como sistema, quiero calcular el total automáticamente para evitar errores.

Pago y Facturación

- Como cajero, quiero seleccionar el método de pago para registrar la venta.
- Como cajero, quiero ingresar el dinero recibido para calcular el cambio.
- Como sistema, quiero calcular el cambio automáticamente.
- Como sistema, quiero generar una factura con los datos de la compra.

Reportes Diarios

- Como administrador, quiero ver la cantidad de ventas del día.
- Como administrador, quiero ver los productos vendidos.
- Como administrador, quiero ver el dinero recibido por tipo de pago.
- Como administrador, quiero ver el cajero responsable del día.

Reportes Mensuales

- Como administrador, quiero ver los ingresos del mes.
- Como administrador, quiero ver los gastos del mes.
- Como administrador, quiero conocer las ganancias o pérdidas del mes.

Diagramas del Sistema

Esta sección presenta y explica los diagramas elaborados para modelar el comportamiento, la estructura y el contexto del sistema POS. Los diagramas se construyeron siguiendo las convenciones del Lenguaje Unificado de Modelado (UML) y sirven como base de comunicación entre el equipo de desarrollo y los interesados del proyecto (Booch et al., 2005).

Diagrama de Casos de Uso

El diagrama de casos de uso representa, a nivel general, las interacciones entre los actores del sistema y las funcionalidades que este provee. En el sistema POS se identificaron dos actores principales: el **Administrador** y el **Cajero**.

El Administrador tiene acceso a las funcionalidades de mayor alcance: consultar el resumen de ventas y gestionar productos e inventario, lo que incluye registrar, editar y eliminar productos. El Cajero, por su parte, opera en el módulo de ventas: gestiona las cuentas de venta, procesa transacciones y emite facturas. Ambos actores comparten el caso de uso de inicio de sesión, que actúa como punto de entrada al sistema y garantiza el control de acceso diferenciado por rol.

Las relaciones de inclusión (*include*) entre los casos de uso de *Gestionar productos e inventario* y las acciones específicas (registrar, editar, eliminar producto) indican que estas últimas son pasos necesarios dentro del flujo principal. Las relaciones de extensión (*extend*) entre *Gestionar cuenta de venta* y *Procesar venta*, así como entre *Procesar venta* y *Emitir factura*, modelan comportamientos opcionales que se activan bajo condiciones específicas durante el proceso de venta.

Profundización del Diagrama de Casos de Uso: Administrador

Este diagrama detalla los flujos internos de las funcionalidades exclusivas del Administrador, descomponiendo cada caso de uso general en sus pasos específicos.

Consulta de Resumen de Ventas. El flujo comienza con el inicio de sesión con usuario administrador. Desde el menú principal, el administrador selecciona la opción *Reportes* y elige el

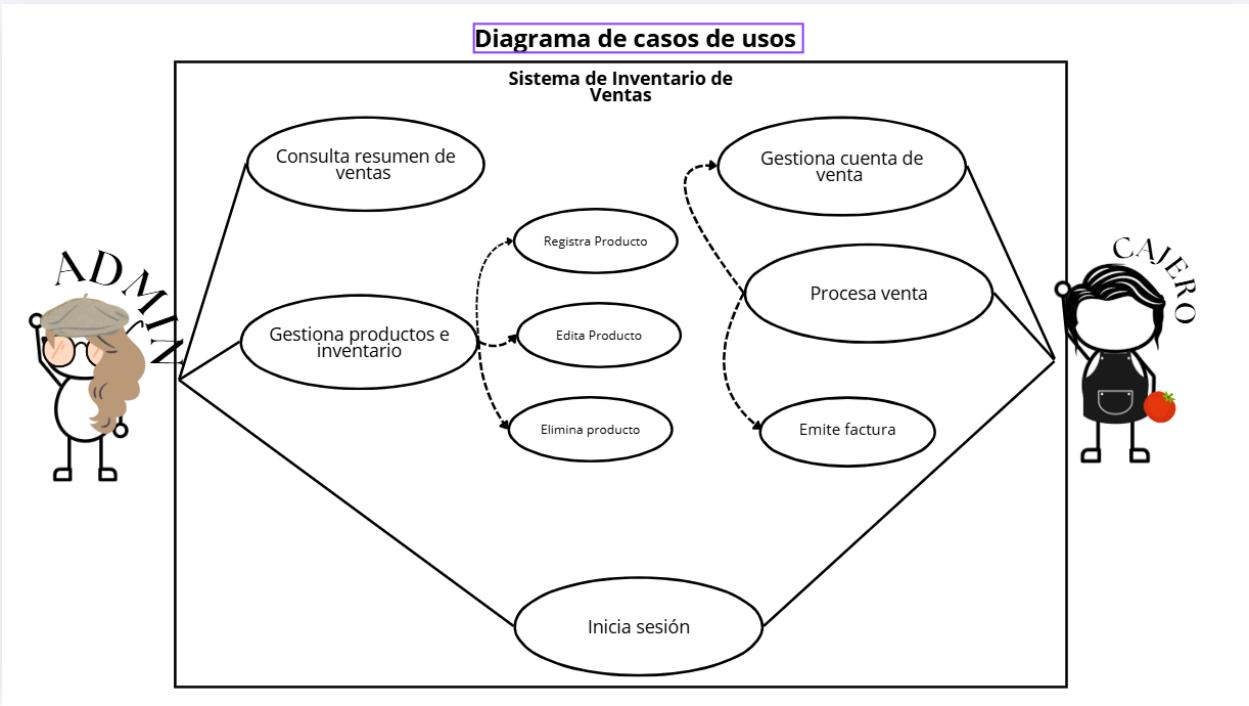

Figura 1: Diagrama de casos de usos

rango de fechas de interés: hoy, ayer, esta semana, este mes o un rango personalizado. El sistema presenta los indicadores principales: total de ventas brutas, descuentos aplicados, impuestos cobrados, ventas netas, número de transacciones y ticket promedio. Adicionalmente, se muestra un gráfico de ventas por período que permite identificar tendencias de rendimiento.

Gestión de Productos e Inventario. Desde el módulo de inventario, el administrador puede realizar tres tipos de operaciones. En primer lugar, la **revisión del inventario**, que permite visualizar el stock disponible, el stock mínimo configurado y el estado de cada producto (Normal, Bajo o Agotado). En segundo lugar, el **ajuste de inventario**, que consiste en seleccionar un producto, elegir el tipo de ajuste, indicar la cantidad real, registrar el motivo (merma, pérdida o conteo físico) y confirmar la operación; el sistema actualiza el stock automáticamente. En tercer lugar, la **edición de productos**, que permite modificar el nombre, precio, costo e imagen de un producto, configurar el stock mínimo y activar alertas de reabastecimiento. Paralelamente, el flujo de **órdenes de compra y entrada manual** permite al administrador seleccionar un proveedor, agregar productos, ingresar las cantidades recibidas, confirmar la recepción y dejar que el sistema actualice

el stock de forma automática.

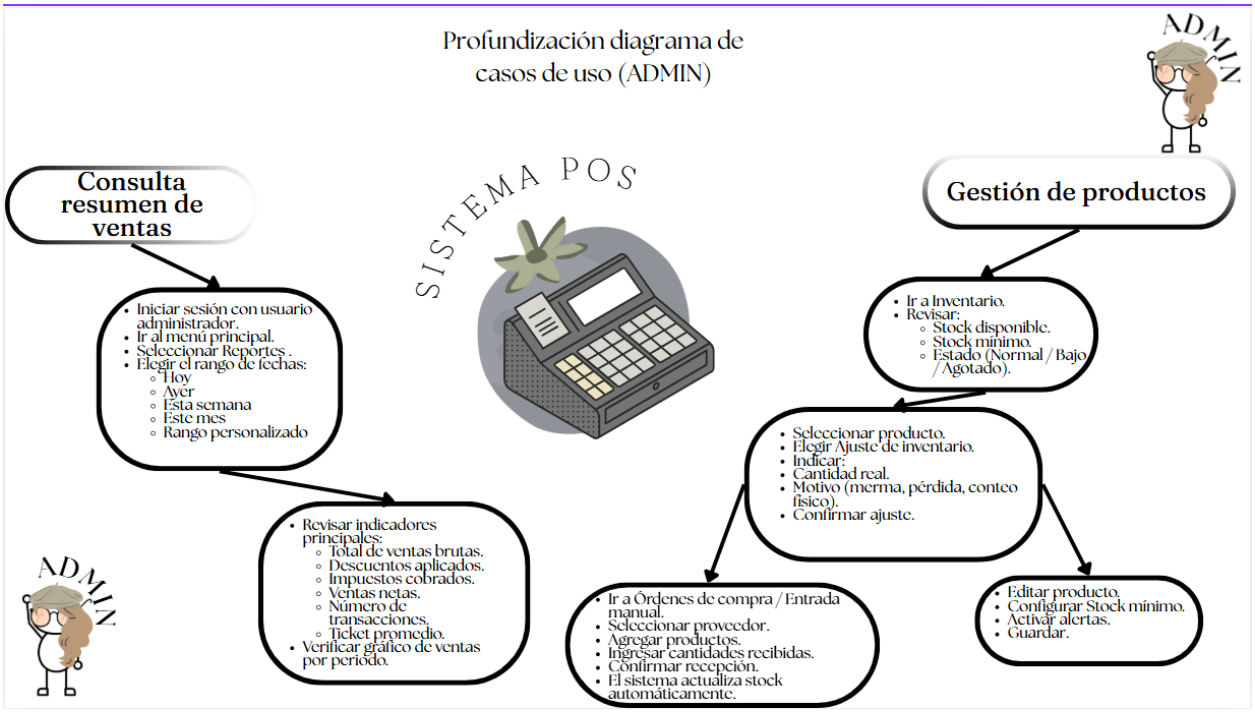

Figura 2: Diagrama de casos de usos: Profundización administrador

Profundización del Diagrama de Casos de Uso: Cajero

Este diagrama detalla los flujos específicos de las tres funcionalidades principales del Cajero: gestión de la cuenta de venta, emisión de factura y procesamiento de la venta.

Gestión de Cuenta de Venta. El cajero ingresa al módulo de ventas y selecciona *Nueva venta* o *Nueva cuenta*. De manera opcional, puede asociar un cliente a la cuenta, ya sea buscando uno existente por nombre, teléfono, cédula o RUT, o creando uno nuevo. Si el sistema lo requiere, se asigna un vendedor y se confirma la apertura de la cuenta.

Emisión de Factura. El cajero puede buscar productos por código de barras, nombre o categoría, seleccionar el producto e ingresar la cantidad deseada. Antes de confirmar, tiene la posibilidad de aplicar un descuento por ítem o de forma global, así como un precio especial si está autorizado para ello. Una vez confirmada la venta, el sistema genera la factura automáticamente

con los datos de la transacción.

Procesamiento de la Venta. Durante el procesamiento, el cajero puede editar la cantidad de cualquier producto en la cuenta, eliminar ítems, cambiar el precio de un producto si cuenta con los permisos correspondientes, agregar observaciones y aplicar un descuento global si aplica. Este flujo garantiza flexibilidad operativa durante la atención al cliente sin comprometer la integridad del registro de la venta.

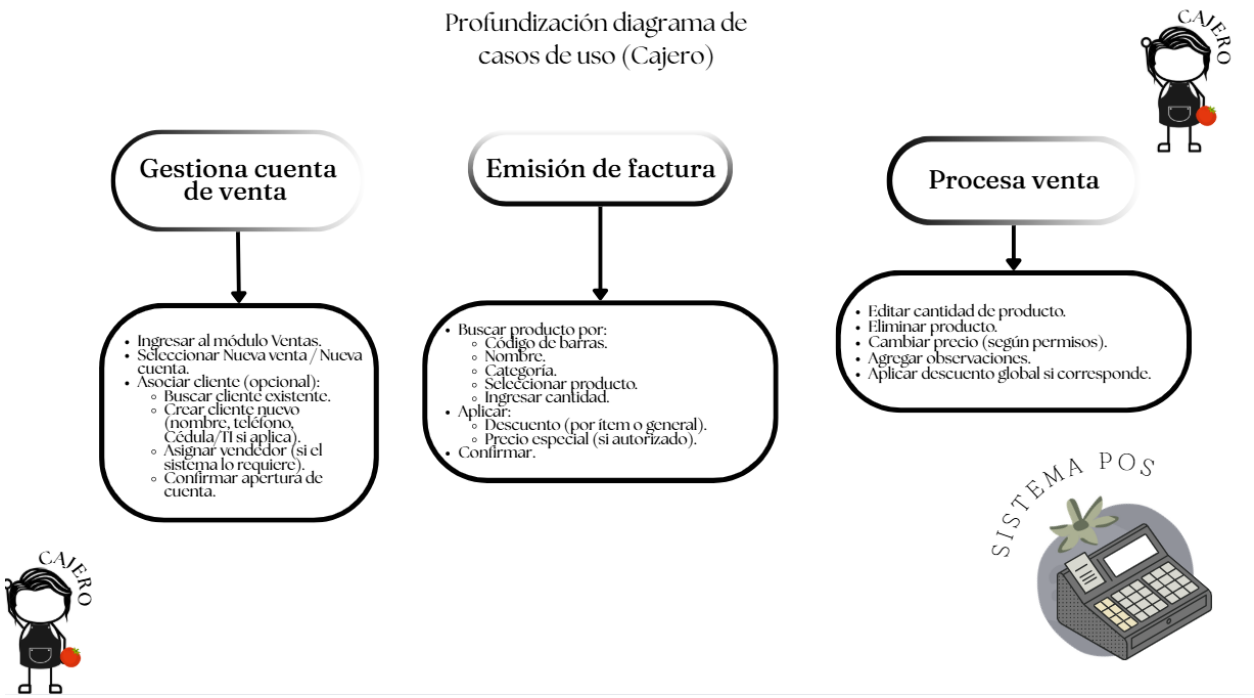

Figura 3: Diagrama de casos de usos: Profundización cajero

Diagrama de Contexto

El diagrama de contexto sitúa al sistema POS dentro de su entorno operativo, mostrando las entidades externas con las que interactúa y los flujos de información que se intercambian. A diferencia del diagrama de casos de uso, el diagrama de contexto no descompone las funcionalidades internas del sistema, sino que lo trata como una caja negra y define sus límites con el mundo exterior (Yourdon, 1989).

Las entidades externas identificadas para el sistema POS son las siguientes. El **Adminis-**

trador envía al sistema solicitudes de consulta de reportes, ajustes de inventario, registro de gastos y configuración de productos; el sistema le devuelve reportes financieros, estados de inventario y confirmaciones de operación. El **Cajero** interactúa con el sistema enviando datos de ventas, selecciones de productos y métodos de pago; el sistema responde con totales calculados, cambio y facturas generadas. El **Cliente** recibe la factura emitida al finalizar la transacción y, opcionalmente, puede ser registrado en el sistema con sus datos de contacto. Finalmente, los **Proveedores** son referenciados en las órdenes de compra y entradas manuales de inventario gestionadas por el administrador.

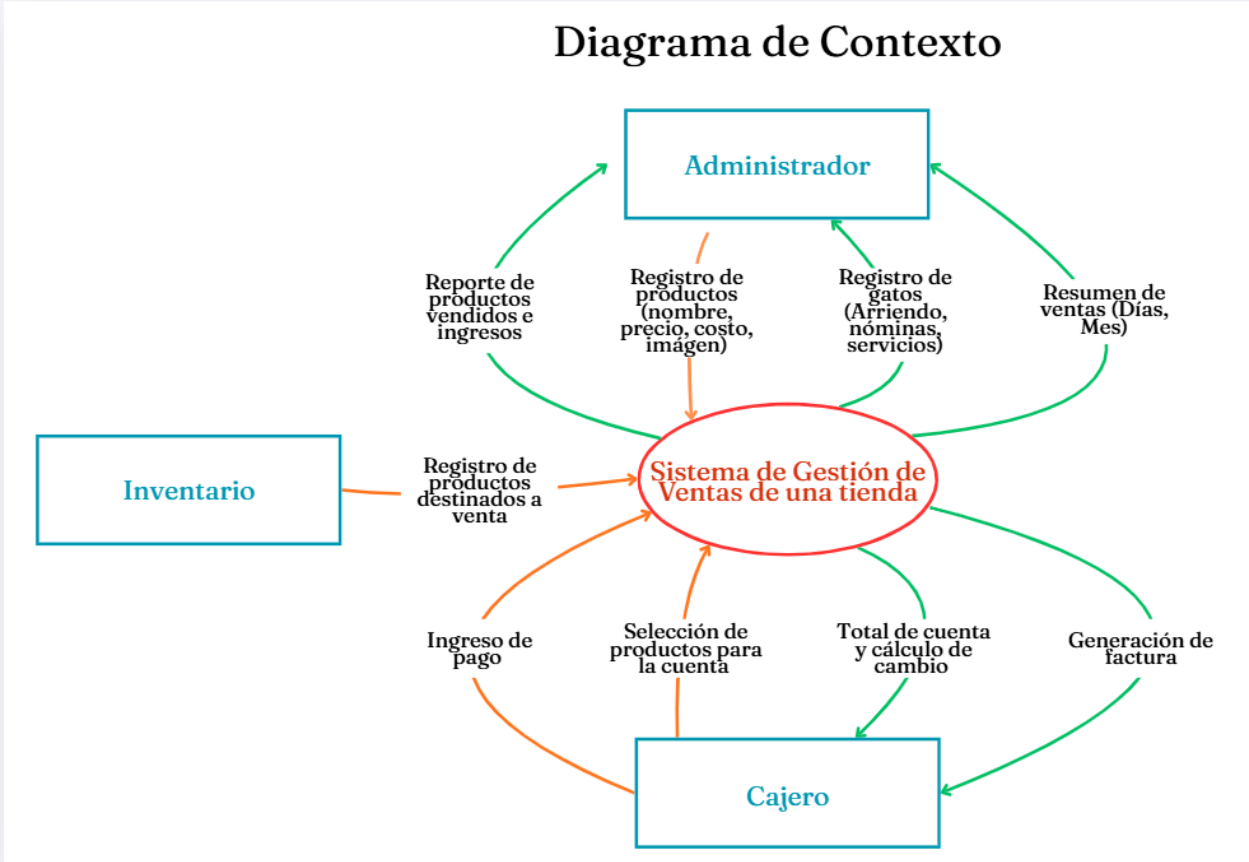

Figura 4: Diagrama de contexto

Diagrama de Secuencias

El diagrama de secuencias modela la interacción temporal entre los objetos del sistema para un escenario específico. A diferencia del diagrama de casos de uso, que muestra *qué* funcionalida-

des existen, el diagrama de secuencia muestra *cómo* se implementa una funcionalidad en términos de mensajes y orden de ejecución (Booch et al., 2005).

Para el sistema POS, se construyó un diagrama de secuencias para el escenario de **registro de una venta completa**. Este escenario fue seleccionado por ser la operación crítica de mayor frecuencia en el negocio y por involucrar la colaboración de múltiples componentes.

El flujo comienza cuando el **Cajero** selecciona *Nueva venta* desde la interfaz. El **Frontend** envía una solicitud POST al **Backend** para crear una venta en estado *pendiente*. El **Backend** inserta un registro en la tabla *Venta* de la **Base de Datos** y devuelve el identificador de la venta.

A continuación, el cajero escanea o busca un producto. El **Frontend** envía la solicitud de agregar producto con el código de barras, la cantidad y el ID de la venta. El **Backend** consulta el producto en la **Base de Datos**, verifica disponibilidad de stock y, si es suficiente, crea un registro en la tabla *DetalleVenta*. Posteriormente, el **Backend** calcula el nuevo total de la venta y actualiza el registro correspondiente. Finalmente, devuelve al **Frontend** el detalle actualizado, que se muestra al cajero.

Este proceso se repite tantas veces como productos se agreguen a la venta. Cuando el cajero confirma la transacción, el **Frontend** envía una solicitud POST para finalizar la venta. El **Backend** inicia una transacción en la **Base de Datos**, actualiza el stock de cada producto afectado, cambia el estado de la venta a *completada*, genera el registro en la tabla *Factura* y confirma la transacción. Finalmente, el **Frontend** recibe los datos de la factura generada y los presenta al cajero para su impresión.

Este diagrama evidencia que la operación de venta involucra cuatro interacciones principales con el backend (crear venta, agregar producto, agregar siguiente producto, confirmar venta) y que la integridad de los datos se garantiza mediante una transacción que actualiza simultáneamente el stock y el estado de la venta.

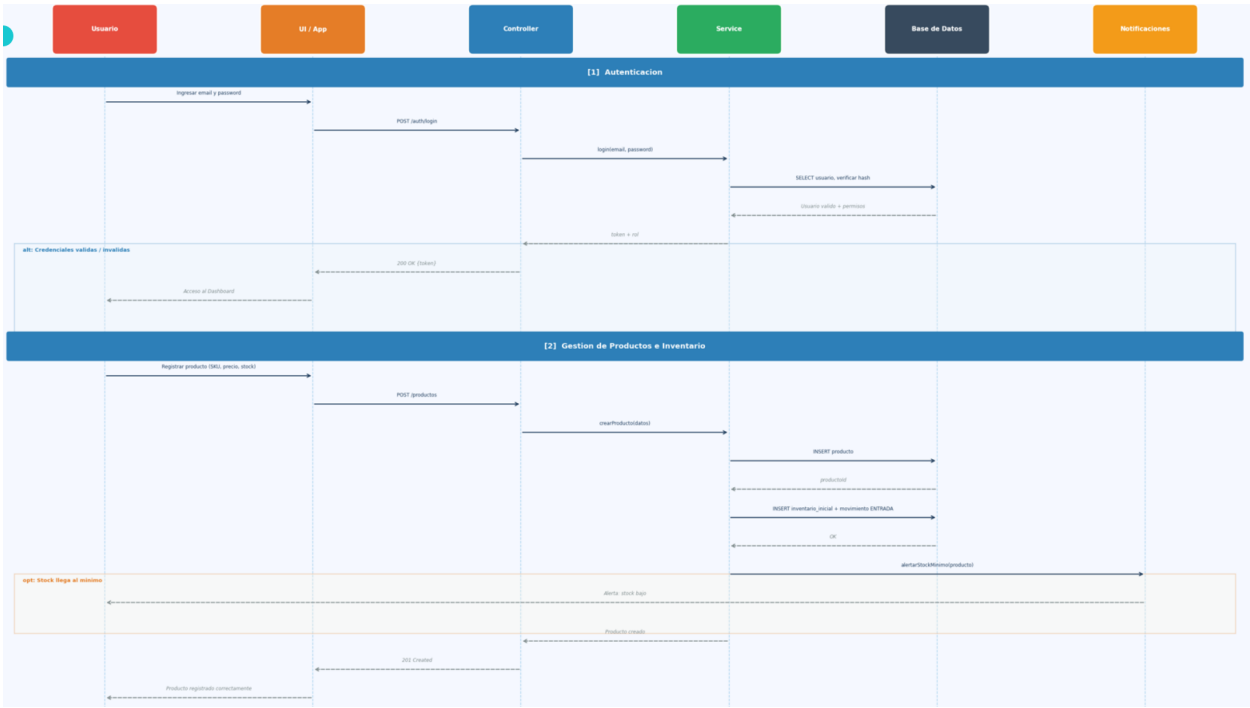

Figura 5: Diagrama de secuencia

Diagrama de Entidad-Relación

El diagrama entidad-relación (DER) constituye el modelo conceptual de la base de datos del sistema. Describe las entidades que almacenarán información, sus atributos y las relaciones entre ellas, independientemente de la tecnología de base de datos que se implemente posteriormente (Elmasri & Navathe, 2016).

Las entidades identificadas para el sistema POS son las siguientes:

Usuario. Representa a todas las personas que pueden acceder al sistema: administrador y cajeros. Sus atributos incluyen identificador (clave primaria), nombre completo, correo electrónico (único), contraseña cifrada, rol (administrador o cajero) y estado de activación.

Producto. Almacena la información de cada artículo del catálogo. Sus atributos son identificador (clave primaria), nombre, precio de venta, costo, URL de imagen, stock actual, stock mínimo y estado (activo o inactivo).

Categoria. Agrupa los productos en familias para facilitar la búsqueda y los reportes. Contiene identificador (clave primaria), nombre y descripción. La relación entre *Categoria* y *Producto*

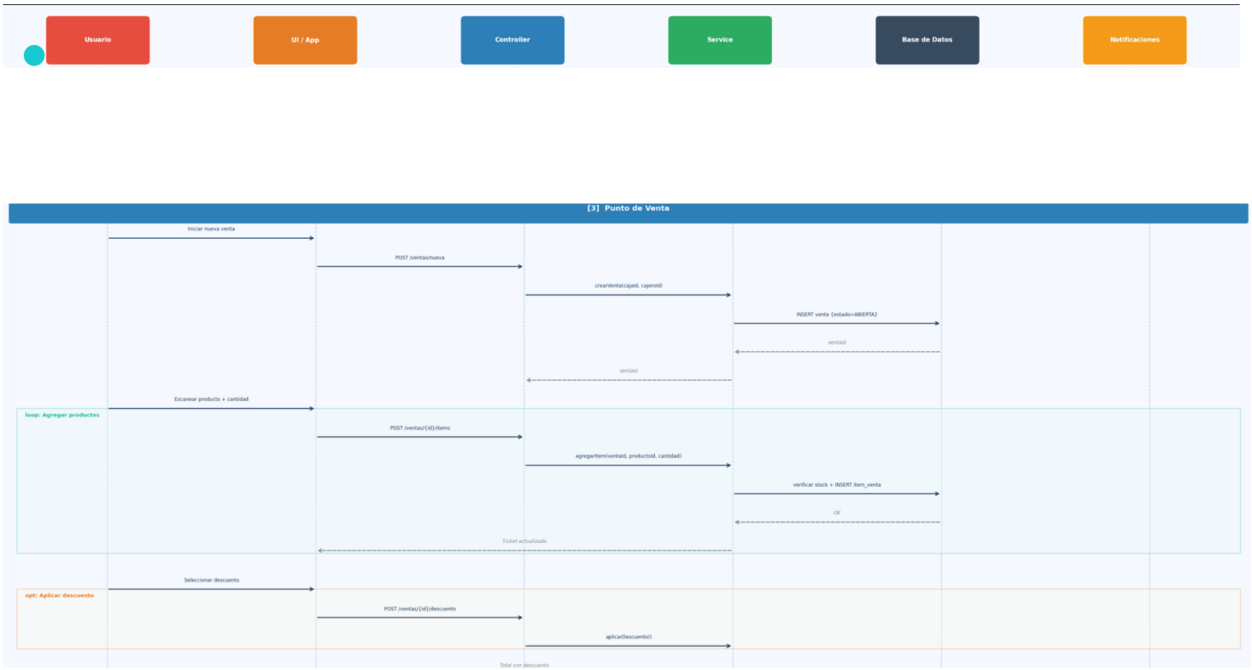

Figura 6: Diagrama de secuencia

es de uno a muchos (una categoría puede tener muchos productos).

Venta. Representa cada transacción comercial realizada. Sus atributos son identificador (clave primaria), referencia al cajero (clave foránea hacia *Usuario*), método de pago, total de la venta, monto recibido, cambio entregado y fecha. La relación con *Usuario* es de muchos a uno (un cajero registra muchas ventas).

DetalleVenta. Descompone cada venta en sus ítems individuales. Es una entidad débil cuya existencia depende de *Venta*. Sus atributos son identificador (clave primaria), referencia a la venta (clave foránea), referencia al producto (clave foránea), cantidad vendida y precio unitario en el momento de la venta. La relación con *Venta* es de muchos a uno (una venta tiene muchos detalles), y con *Producto* también es de muchos a uno (un producto aparece en muchos detalles).

Factura. Almacena el comprobante generado tras cada venta. Sus atributos son identificador (clave primaria), número de factura (único), referencia a la venta (clave foránea), fecha de emisión y datos del cliente. La relación con *Venta* es de uno a uno (cada venta genera una factura).

Gasto. Registra los egresos del negocio. Sus atributos son identificador (clave primaria), referencia al usuario que registró el gasto (clave foránea), descripción, monto, categoría (servicios,

reposición, mantenimiento, etc.) y fecha.

MovimientoInventario. Trazabiliza los ajustes manuales al stock que no provienen de ventas (mermas, pérdidas, conteos físicos). Sus atributos son identificador (clave primaria), referencia al producto (clave foránea), referencia al usuario que realizó el ajuste (clave foránea), tipo de movimiento (incremento o decremento), cantidad, motivo y fecha.

El modelo relacional derivado de este diagrama garantiza integridad referencial mediante las claves foráneas y permite consultas eficientes para los reportes financieros y de inventario.

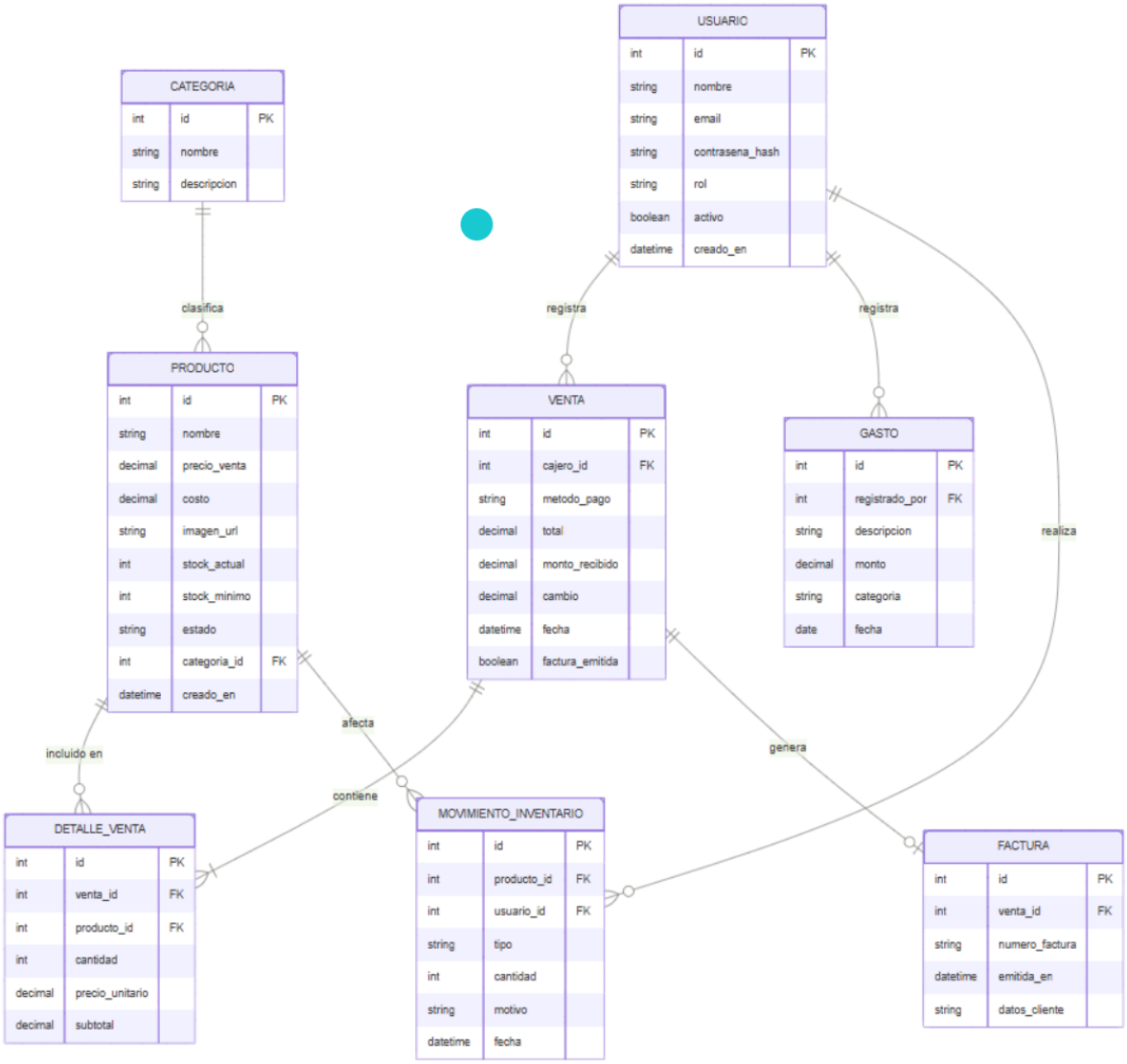

Figura 7: Diagrama de entidad relación

Diagrama de Clases UML

El diagrama de clases UML modela la estructura estática del sistema, definiendo las clases que lo componen, sus atributos, métodos y las relaciones entre ellas. Este diagrama constituye la base del diseño orientado a objetos del sistema y se corresponde directamente con el modelo de entidad-relación de la base de datos (Booch et al., 2005).

Las clases principales identificadas son las siguientes:

Usuario representa tanto al administrador como al cajero. Sus atributos incluyen identificador, nombre, correo electrónico, contraseña cifrada, rol y estado de activación. Sus métodos principales son `iniciarSesion()`, `cerrarSesion()` y `validarRol()`.

Producto almacena la información de cada artículo del catálogo: identificador, nombre, precio de venta, costo, URL de imagen, stock actual, stock mínimo, estado y categoría asociada. Sus métodos incluyen `actualizarStock()`, `verificarDisponibilidad()` y `activarAlerta()`.

Categoria agrupa los productos en familias para facilitar la búsqueda. Contiene identificador, nombre y descripción, y se relaciona con *Producto* mediante una asociación de composición (una categoría puede contener muchos productos).

Venta representa cada transacción comercial. Sus atributos son identificador, referencia al cajero, método de pago, total, monto recibido, cambio y fecha. Sus métodos son `calcularTotal()`, `calcularCambio()` y `generarFactura()`. Se relaciona con *Usuario* (el cajero que la registra), con *DetalleVenta* (composición: una venta contiene uno o más detalles) y con *Factura* (asociación: una venta genera una factura).

DetalleVenta es la clase intermedia que descompone cada venta en sus ítems individuales. Contiene identificador, referencia a la venta, referencia al producto, cantidad, precio unitario y subtotal. Su método principal es `calcularSubtotal()`.

Factura representa el comprobante generado tras cada venta. Sus atributos son identificador, número de factura, fecha de emisión y datos del cliente. Su método es `imprimir()`.

Gasto registra los egresos del negocio. Contiene identificador, referencia al usuario que lo registró, descripción, monto, categoría y fecha. Su método es `registrar()`.

MovimientoInventario trazabiliza los ajustes manuales al stock. Sus atributos son identificador, referencias al producto y al usuario, tipo de movimiento, cantidad, motivo y fecha. Su método es `aplicarAjuste()`.

Las relaciones entre clases siguen los principios de la orientación a objetos: *Usuario* se asocia con *Venta* (un cajero registra muchas ventas), con *Gasto* (un usuario registra muchos gastos) y con *MovimientoInventario* (un usuario realiza muchos movimientos). *Producto* se asocia con *DetalleVenta* (un producto aparece en muchos detalles) y con *MovimientoInventario* (un producto tiene muchos movimientos). *Categoria* se compone de muchos *Productos*.

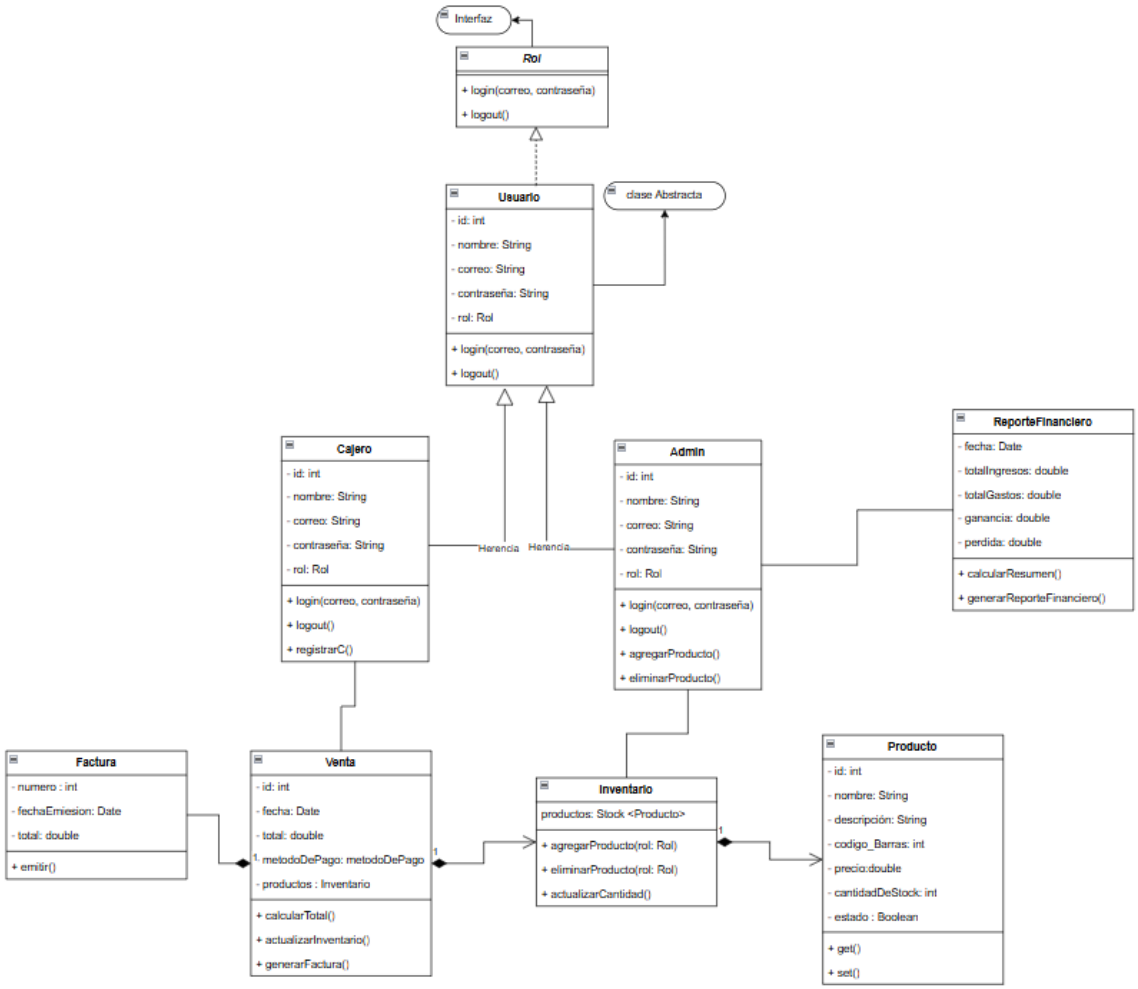

Figura 8: Diagrama de clases

Metodología de Desarrollo

Para la gestión del proyecto se realizó un análisis comparativo de tres metodologías ágiles ampliamente utilizadas en el desarrollo de software: Scrum, Kanban y Extreme Programming (XP).

Resumen de Metodologías Evaluadas

Scrum. Scrum es un marco de trabajo iterativo e incremental que organiza el trabajo en ciclos de tiempo fijos denominados *sprints*, generalmente de dos a cuatro semanas (Schwaber & Sutherland, 2020). Sus principios fundamentales son el compromiso del equipo con los objetivos del sprint, el enfoque en las tareas del período, la apertura en la comunicación del progreso, el respeto entre los integrantes y el coraje para enfrentar problemas. El marco se sustenta en los pilares del empirismo: transparencia, inspección y adaptación, promoviendo la autoorganización del equipo.

Kanban. Kanban es un método de gestión del flujo de trabajo de origen japonés, adaptado al desarrollo de software por Anderson (2010). Sus principios fundamentales incluyen comenzar con el proceso actual sin imponer cambios radicales, aceptar la mejora evolutiva e incremental, respetar los roles y responsabilidades existentes, y fomentar el liderazgo distribuido en todos los niveles del equipo. El método se basa en la visualización del trabajo mediante un tablero, la limitación del trabajo en progreso (WIP) y la gestión del flujo continuo.

Extreme Programming (XP). XP es una metodología ágil orientada a la excelencia técnica, propuesta por Beck (2000). Sus prácticas incluyen el juego de planificación colaborativa entre cliente y equipo, entregas pequeñas y frecuentes, diseño simple, pruebas automatizadas, refactorización continua, programación en pareja, propiedad colectiva del código, integración continua, ritmo sostenible, presencia del cliente en sitio y estándares de codificación compartidos.

Metodología Implementada

Selección de Kanban. Después de analizar las tres metodologías en función del contexto del proyecto (equipo de tres estudiantes, semestre académico, sistema de alcance moderado), el equipo seleccionó **Kanban** como metodología de desarrollo. La decisión se fundamentó en los siguientes criterios:

En primer lugar, Kanban no requiere roles estrictos ni iteraciones obligatorias, lo que permite a los tres integrantes contribuir simultáneamente en diferentes módulos del sistema (inventario, ventas y reportes) sin restricciones de asignación. En segundo lugar, la visualización del trabajo mediante un tablero facilita el seguimiento del avance de forma transparente y sencilla. En tercer lugar, el principio de limitación del trabajo en progreso (WIP) evita la dispersión del equipo y garantiza que las tareas iniciadas se completen antes de comenzar nuevas. Finalmente, la flexibilidad de Kanban resulta especialmente adecuada para un proyecto académico donde los tiempos disponibles varían semana a semana.

En contraste, XP exige prácticas como el desarrollo guiado por pruebas (TDD) y la programación en pareja que, si bien son valiosas, representan una carga metodológica difícil de sostener en el contexto universitario. Scrum, por su parte, requiere ceremonias formales (dailies, revisiones, retrospectivas) y roles definidos (Product Owner, Scrum Master) que añaden complejidad organizativa innecesaria para un equipo pequeño.

Roles del Equipo de Desarrollo. Frontend. Encargado de la interfaz visual del sistema. Diseña y desarrolla las pantallas que utilizarán el administrador y el cajero, incluyendo el inicio de sesión, el panel de productos, la creación de cuentas, el proceso de venta y la visualización de reportes. Garantiza que el sistema sea intuitivo, rápido y correctamente integrado con el backend.

Backend. Responsable de la lógica del sistema y la gestión de datos. Desarrolla el sistema de autenticación, la gestión de productos, el control de inventario, el procesamiento de ventas, la generación de facturas y los reportes. Administra la base de datos y asegura la corrección de los cálculos financieros.

Tester. Encargado de verificar que el sistema funcione correctamente. Realiza pruebas sobre el inicio de sesión, el registro de productos, las ventas, los cálculos, la generación de facturas y los reportes. Identifica errores, valida el cumplimiento de los requerimientos y asegura una experiencia de usuario adecuada antes de la entrega.

Arquitectura del Sistema

Decisión Arquitectónica

El equipo seleccionó una **arquitectura monolítica modular** para el sistema de gestión POS. Esta decisión responde al contexto operativo del sistema: un único local físico, un administrador y entre uno y tres cajeros, con un volumen de transacciones moderado que no requiere alta escalabilidad ni infraestructura distribuida.

Implementar una arquitectura de microservicios resultaría innecesaria y sobredimensionada para el alcance del proyecto. En cambio, la arquitectura monolítica modular permite desarrollar una aplicación única organizada internamente por responsabilidades, lo que garantiza simplicidad, eficiencia y estabilidad operativa.

Organización Modular

La solución consiste en un único despliegue con separación lógica clara entre los siguientes módulos:

- **Módulo de usuarios:** gestión de autenticación y control de acceso por rol.
- **Módulo de productos:** CRUD de productos y gestión de categorías.
- **Módulo de inventario:** control de stock, ajustes manuales y alertas.
- **Módulo de ventas:** procesamiento de transacciones y generación de facturas.
- **Módulo de reportes:** reportes diarios, mensuales y comparativos.
- **Módulo de gastos:** registro y consulta de egresos del negocio.

Esta estructura proporciona orden interno, mejor mantenibilidad, bajo costo de implementación y facilidad de administración. Aunque la arquitectura presenta una escalabilidad más limitada en comparación con los microservicios, la organización modular interna permite separar responsabilidades y facilitar futuras mejoras sin incorporar complejidad técnica innecesaria.

Plan de Pruebas

Para garantizar la calidad del sistema POS, el equipo definió un plan de pruebas basado en las historias de usuario y los requerimientos funcionales identificados. Las pruebas se organizan en cinco tipos complementarios, aplicables dentro del flujo Kanban adoptado. Cada tarea del tablero se considera completada (*Done*) únicamente cuando pasa las pruebas funcionales y unitarias correspondientes.

Pruebas Unitarias

Las pruebas unitarias verifican el correcto funcionamiento de funciones o métodos individuales de forma aislada, sin depender de otras capas del sistema. Son la primera línea de defensa ante errores lógicos (Beck, 2003).

Definición: prueba que evalúa una unidad mínima de código (función, método o clase) de forma independiente, con entradas controladas y salidas verificables.

Ejemplos aplicados al sistema POS:

- `calcularCambio(totalVenta, montoRecibido)`: verifica que el cambio sea correcto para distintos montos.
- `calcularSubtotal(precio, cantidad)`: verifica el resultado para distintas cantidades, incluyendo casos borde (cantidad cero, precio negativo).
- `validarCredenciales(email, hash)`: verifica que retorne verdadero o falso según el caso.
- `actualizarStock(productoId, cantidad)`: verifica que el stock disminuya correctamente tras una venta.

Herramientas sugeridas: Jest (JavaScript/Node.js), Vitest (entornos Vite), PyTest (backend Python).

Criterios de aceptación:

- Todas las funciones críticas de negocio tienen al menos un caso positivo y uno negativo.
- El 100 % de los cálculos financieros (total, cambio, subtotal, ganancia) pasan las pruebas.

- Las pruebas son reproducibles y no dependen del estado de la base de datos.

Pruebas de Integración

Las pruebas de integración verifican que dos o más módulos del sistema funcionen correctamente cuando se combinan. En la arquitectura monolítica modular del proyecto, esto implica probar la comunicación entre el frontend, el backend y la base de datos (Sommerville, 2016).

Definición: prueba que evalúa la interacción entre módulos o capas del sistema para detectar fallos en la comunicación, el intercambio de datos o las dependencias entre componentes.

Ejemplos aplicados al sistema POS:

- Registrar una venta desde el módulo de ventas y verificar que el inventario se actualice automáticamente.
- Crear un producto desde el panel de administración y verificar que aparezca en la búsqueda del cajero.
- Completar una venta y comprobar que se genere correctamente el registro en la tabla FACTURA.
- Registrar un gasto y verificar que el reporte mensual lo refleje.

Herramientas sugeridas: Supertest (pruebas de API REST en Node.js), Postman/Newman (automatización de colecciones), pytest + requests (Python).

Criterios de aceptación:

- La actualización de inventario ocurre en la misma transacción que la venta, sin inconsistencias.
- Los datos persisten correctamente en la base de datos tras cada operación.
- Los endpoints de la API devuelven los códigos HTTP correctos (200, 201, 400, 401, 404).

Pruebas Funcionales

Las pruebas funcionales validan que el sistema cumple con los requerimientos especificados en las historias de usuario. Cada historia equivale a un caso de prueba funcional que debe pasar antes de considerarse completada (Cohn, 2004).

Definición: prueba que verifica que una funcionalidad del sistema se comporta exactamente como se especificó en los requerimientos, desde la perspectiva del usuario.

Ejemplos aplicados al sistema POS:

- HU-Auth-01: el administrador puede iniciar sesión con credenciales válidas y accede al panel de administración.
- HU-Venta-03: el cajero puede ver el total actualizado al agregar o quitar productos de la cuenta.
- HU-Pago-04: el sistema calcula el cambio automáticamente al ingresar el monto recibido.
- HU-Reporte-01: el administrador puede filtrar el resumen de ventas por día, semana o mes.
- HU-Inv-02: el inventario se descuenta automáticamente tras confirmar una venta.

Herramientas sugeridas: Cypress (pruebas E2E sobre la interfaz web), Playwright, Cucumber (BDD con escenarios en lenguaje natural).

Criterios de aceptación:

- Cada historia de usuario tiene al menos un escenario de prueba documentado.
- Los flujos principales (inicio de sesión $\rightarrow$ venta $\rightarrow$ factura) pasan sin errores en un entorno de prueba.
- Las restricciones de roles se cumplen: el cajero no puede acceder a reportes financieros.

Pruebas de Usabilidad

Las pruebas de usabilidad evalúan qué tan fácil e intuitivo resulta el sistema para los usuarios finales. Dado que el sistema operará en un entorno de tienda física con ritmo de trabajo rápido, la usabilidad es un requerimiento no funcional crítico (Nielsen, 1994).

Definición: evaluación de la facilidad de uso, claridad de la interfaz y eficiencia con que los usuarios pueden completar tareas sin asistencia externa ni errores frecuentes.

Ejemplos aplicados al sistema POS:

- Un cajero sin experiencia previa puede completar una venta completa en menos de dos minutos.

- El administrador puede encontrar el reporte mensual en máximo tres clics desde el inicio.
- Los mensajes de error son comprensibles (por ejemplo, “Stock insuficiente” en lugar de códigos internos).
- El sistema es operable en tableta o celular sin pérdida de funcionalidad crítica.

Herramientas sugeridas: pruebas manuales con usuarios reales, Google Forms para recolectar retroalimentación, Loom para grabar sesiones de prueba.

Criterios de aceptación:

- Al menos un usuario externo completa los flujos principales sin asistencia del desarrollador.
- No se presentan errores de interfaz que bloqueen completar una venta.
- El tiempo promedio para registrar una venta sencilla es menor a 90 segundos.

Pruebas de Seguridad

Las pruebas de seguridad verifican que el sistema protege correctamente los datos y restringe el acceso según los roles definidos (OWASP, 2021).

Definición: prueba que evalúa si el sistema resiste accesos no autorizados, protege datos sensibles y aplica correctamente las restricciones de roles y permisos.

Ejemplos aplicados al sistema POS:

- Un cajero que intenta acceder a `/admin/reportes` recibe una respuesta HTTP 403 Forbidden.
- Las contraseñas se almacenan cifradas (bcrypt u otro algoritmo de hash) y nunca en texto plano.
- Un token de sesión expirado no permite realizar operaciones sin volver a autenticarse.
- No es posible modificar el precio de un producto desde la vista del cajero mediante inspección del navegador.

Herramientas sugeridas: pruebas manuales de control de acceso, OWASP ZAP (análisis básico de vulnerabilidades web), revisión de cabeceras HTTP.

Criterios de aceptación:

- El sistema rechaza toda solicitud sin token válido con código HTTP 401.
- Ningún rol accede a funcionalidades fuera de su alcance definido en los requerimientos.
- Las contraseñas no son legibles en la base de datos ni en los registros del sistema.

Resumen del Plan de Pruebas

Cuadro 1: *Resumen del plan de pruebas del sistema POS*

Tipo de prueba	Responsable	Herramienta principal	Prioridad
Unitarias	Backend / Frontend	Jest / PyTest	Alta
Integración	Backend	Supertest / Postman	Alta
Funcionales	Tester	Cypress / Manual	Alta
Usabilidad	Tester	Usuario real	Media
Seguridad	Backend / Tester	Manual / OWASP ZAP	Media

Mockup

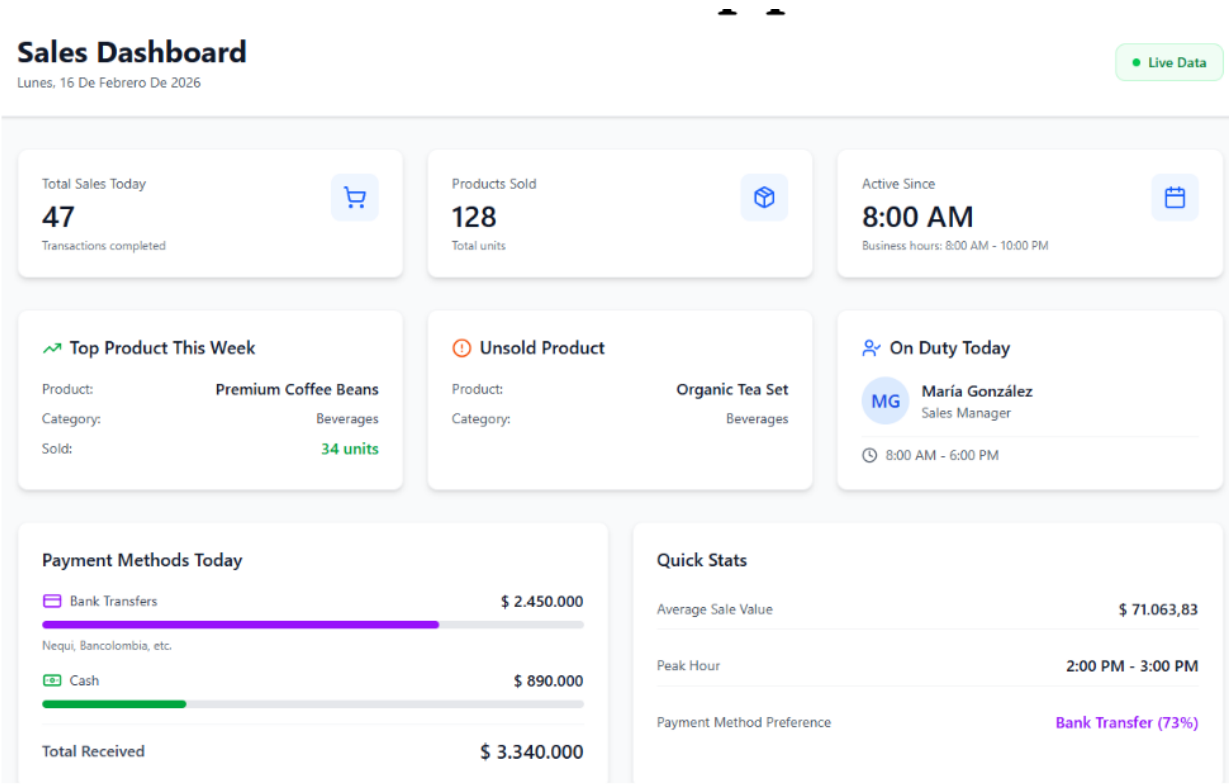

Figura 9: Mockup Resumen de Ventas

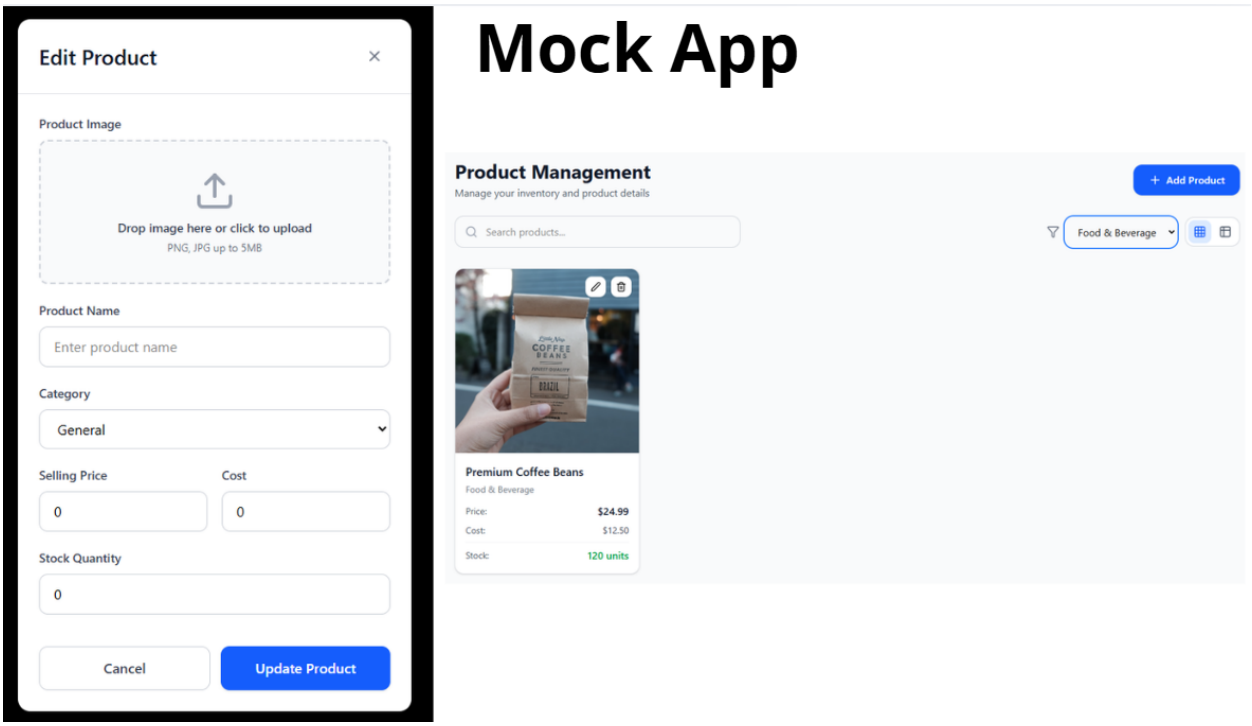

Figura 10: Mockup Productos

Referencias

Anderson, D. J. (2010). *Kanban: Successful evolutionary change for your technology business*. Blue Hole Press.

Beck, K. (2000). *Extreme programming explained: Embrace change*. Addison-Wesley.

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The unified modeling language user guide* (2.a ed.). Addison-Wesley.

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.

OWASP Foundation. (2021). *OWASP top ten*. `https://owasp.org/www-project-top-ten/`

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. `https://scrumguides.org`

Sommerville, I. (2016). *Software engineering* (10.a ed.). Pearson.

Yourdon, E. (1989). *Modern structured analysis*. Prentice Hall.